TRANSURBAN — guia de apresentação no auditório, sem slides

Apresentação de 20/10/2026. Usar o Swagger e, brevemente, o VS Code. Duração sugerida: 10 a 12 minutos, além de perguntas. Os tempos dependem do ensaio. A divisão abaixo distribui as falas; não atribui autoria individual do código.

1. Decisão para a demonstração

A opção recomendada é levar o notebook Windows do líder, com Python 3.12, dependências instaladas, API ensaiada e a fonte de alimentação. Conectar o notebook ao cabo do projetor, se a escola permitir. Assim não é necessário instalar a API no computador do auditório. Uma saída HDMI pode conectar diretamente; USB-C exige suporte a vídeo e adaptador compatível. Peça ao responsável para conferir o cabo e a entrada disponíveis.

Na apresentação: SQLite local para os cadastros DS, CSV público histórico de Curitiba para IA e Swagger servido pelo próprio projeto. A base sintética anterior fica separada no grupo “IA — Base sintética anterior (testes)” e não é o conjunto apresentado nesta revisão. O inicializador iniciar_apresentacao.py define a conexão local no processo; não precisa de .env. O banco local é um banco de dados SQL em arquivo, com PK e FK; não é um banco Supabase. O arquivo .env é configuração privada para outras formas de iniciar, não um requisito para esta demonstração.

A estrutura do Supabase foi conferida por capturas enviadas pelo usuário, incluindo PK/FK, geração automática de IDs e RLS. A conexão Python com o pooler foi interrompida antes da autenticação. Não há comprovação de POST/GET remoto. Não tentar resolver senhas, rede ou TLS no palco.

2. Preparação no Windows, antes de sair de casa

Baixe a versão atual da branch entrega/escola-apresentacao-2026-10-20 e extraia o ZIP. Não use uma cópia antiga sem swagger_static. Abra o terminal do VS Code na pasta backend_escola. Não copie o ambiente virtual Linux para Windows.

Execute um comando de cada vez e confira o resultado antes de continuar:

```
py -3.12 iniciar_apresentacao.py --preparar
py -3.12 iniciar_apresentacao.py --testar
py -3.12 iniciar_apresentacao.py --evidencias
py -3.12 iniciar_apresentacao.py
```

Atalhos opcionais no Windows: na pasta backend_escola, PREPARAR-WINDOWS.cmd executa instalação, testes e evidências e para se houver falha. APRESENTAR-WINDOWS.cmd inicia a API já preparada. Os atalhos foram fornecidos para conveniência; a execução Windows precisa ser ensaiada nesse notebook.

A preparação requer internet e Python 3.12. Os testes devem passar sem falhas; o número de referência nesta revisão é 29 testes. O aviso de depreciação de Starlette/httpx, se aparecer, não é falha. O gerador deve indicar 22 requisições verificadas. A contagem deve ser confirmada no notebook: a validação na nuvem não substitui a execução nele.

Aguarde Application startup complete, deixe o terminal aberto e digite no navegador DO MESMO NOTEBOOK:

- http://127.0.0.1:8000/health: deve mostrar status ok e banco sqlite_local.

- http://127.0.0.1:8000/docs: Swagger para os testes.

- /painel/: painel opcional; não é necessário na apresentação.

O inicializador ativa o modo de apresentação: a documentação /docs mostra o roteiro, DS e os dados públicos; não lista as rotas sintéticas anteriores. Elas continuam disponíveis para os testes, sem serem apagadas. GET /apresentacao mostra a sequência e os resultados esperados.

A partir desta revisão, /docs carrega JS e CSS locais de swagger_static, em vez de CDN. Depois da instalação, faça um ensaio com o Wi-Fi desligado: abra uma nova aba, carregue /docs e execute /health e /ia/dados-publicos/resumo. O banco e o CSV devem funcionar sem internet. Ligue a rede novamente se precisar baixar ou enviar arquivos; não execute instalação durante a apresentação.

Pare com Ctrl+C, inicie novamente com o último comando e faça GET /linhas. Confira que os cadastros do ensaio permaneceram no arquivo apresentacao_local.db. Não apague esse arquivo sem necessidade. IDs podem mudar ao repetir cadastros; use sempre o ID realmente retornado.

Leve também o ZIP atualizado num pendrive, o roteiro e os JSONs das evidências. Como a instalação de dependências exige internet, um pendrive com o código sozinho não garante que um computador novo esteja preparado.

3. Projetor e organização do palco

1. Chegue antes do público. Peça ao responsável para conectar a saída do notebook à entrada correta do projetor. Não altere instalações ou cabeamento fixo sem orientação.

2. No Windows, pressione Win+P e escolha Duplicar. O telão mostrará a mesma tela do notebook. Se não aparecer, confira com o responsável a entrada selecionada e o adaptador; o fato de o cabo entrar não garante compatibilidade.

3. Conecte o carregador, desative a suspensão durante a apresentação e silencie notificações.

4. Inicie a API e abra /docs no navegador do notebook. Não use o celular para abrir 127.0.0.1: ele aponta para o próprio aparelho.

5. Ajuste o zoom do Chrome, por exemplo para 125% ou 150%, e confira a leitura no fundo da sala. Use F11 se ajudar. Evite mostrar páginas longas de JSON sem indicar o campo explicado.

6. Deixe database.py, models.py, schemas.py e main.py abertos no VS Code. Nunca projete .env, senhas ou autenticações.

7. José Henrique pode operar o computador durante todas as falas. Bruno e Kauan apenas falam e podem ler suas frases. Cada colega espera o resultado da tela antes de anunciar o que aconteceu.

8. Deixe os exemplos deste roteiro em um arquivo de texto local. Copie somente o JSON necessário, com calma, sem editar código no palco.

Se tiverem de usar o computador Linux do auditório, preparem-no antes da entrada do público, com Python 3.12. Na pasta backend_escola:

```
python3 iniciar_apresentacao.py --preparar
python3 iniciar_apresentacao.py --testar
python3 iniciar_apresentacao.py --evidencias
python3 iniciar_apresentacao.py
```

Confira antes que python3 --version indica 3.12; não execute esses comandos às cegas em uma versão diferente. Não transfira .venv-apresentacao do Windows para Linux. Use o mesmo

navegador e a mesma URL no computador que estiver executando a API.

4. Ordem e apoio de cada integrante

Ordem	Integrante	Responsabilidade	Apoio na tela
Abertura	José Henrique, líder	Resumo do projeto e limites da demonstração	Swagger aberto, sem clicar
1	Bruno	Objetivo, fala curta	Não opera o computador
2	Kauan	GET, fala curta	Não opera o computador
3	José Henrique	Arquivos, PK/FK, POST e GET	VS Code, depois Swagger
4	Gustavo	Validações e erros esperados	404, consulta e 422
5	Jullyany	CSV e qualidade	GET /ia/qualidade
6	Camila	Indicadores e agrupamentos	Resumo, linha e data
7	Ana Carolina	Evidências, GitHub e conclusão	Roteiro ou resultados já exibidos

Abertura — José Henrique, líder (cerca de 1 minuto e meio)

“Bom dia, pessoal! Eu sou o José Henrique e hoje nossa equipe vai apresentar o TRANSURBAN. Estou aqui com Bruno, Kauan, Gustavo, Jullyany, Camila e Ana Carolina.

Nosso projeto se chama TRANSURBAN e tem como tema a mobilidade e o transporte coletivo entre Maringá e Sarandi. A proposta do projeto é organizar informações que possam apoiar quem utiliza ou estuda esse transporte.

Hoje vamos apresentar o backend, que é a parte do sistema responsável por receber solicitações, validar informações, trabalhar com o banco de dados e devolver resultados. A tela que vocês verão é o Swagger, nossa ferramenta para demonstrar e testar a API. Ela não é a interface final dos passageiros.

A apresentação une duas partes da avaliação. Em Desenvolvimento de Sistemas, vamos cadastrar cidades e linhas de ônibus, mostrar a relação entre elas e testar dados válidos e inválidos. Em Ciência de Dados, vamos verificar um arquivo CSV e mostrar contagens, médias, mediana e agrupamentos disponibilizados pela API.

Nesta demonstração usamos um banco local nos cadastros e uma base pública de programação de transporte de Curitiba na análise. Esse conjunto foi publicado por um projeto comunitário, não é um feed oficial da URBS e se refere a um período histórico. Usamos como estudo externo do TRANSURBAN; não são dados de Maringá e Sarandi. Não há medição de atrasos nem comprovação de viagens realizadas. Vamos mostrar o que está implementado, a origem de cada dado e o que os resultados permitem concluir.

O Bruno vai apresentar o objetivo da demonstração.”

Parte 1 — Bruno (20 a 30 segundos)

“Nosso objetivo é mostrar que a API consegue cadastrar e consultar cidades e linhas de ônibus. Também vamos mostrar como ela rejeita informações inválidas e fornece resultados da análise dos dados. Agora o Kauan vai explicar a consulta.”

Bruno pode ler. Não precisa explicar Python, chaves, bibliotecas ou executar comandos.

Parte 2 — Kauan (20 a 30 segundos)

“GET é o comando usado para consultar informações. Na nossa API, podemos consultar as cidades e as linhas cadastradas. A consulta mostra os dados e não faz um novo cadastro. Agora o José Henrique vai demonstrar.”

Kauan pode ler. Não precisa operar o computador nem explicar os arquivos.

Parte 3 — José Henrique: estrutura e cadastro (cerca de 3 minutos)

Antes de começar os testes, abra GET /apresentacao para conferir a sequência; depois recolha essa operação para dar espaço à demonstração.

Passo 1: mostrar os arquivos no VS Code

“Usamos Python com FastAPI para disponibilizar as rotas. O SQLAlchemy permite trabalhar com as tabelas por meio de objetos Python, e o Pydantic valida os dados recebidos.

O database.py configura a conexão e as sessões de banco. O models.py representa as tabelas. O schemas.py define a validação e o formato das respostas. O main.py registra as rotas e coordena as operações. A parte de Ciência de Dados está em analise.py e ia_routes.py.

Cidade possui id_cidade como chave primária. Linha possui id_linha como chave primária e id_cidade como chave estrangeira. Uma cidade pode ter várias linhas; cada linha precisa apontar para uma cidade existente. Adaptamos o exemplo didático de Categoria e Produto ao contexto do nosso projeto.”

Passo 2: voltar ao Swagger e conferir /health

Abra GET /health, clique em Try it out e depois Execute. Aguarde a resposta.

“O status ok confirma a consulta de verificação ao banco. O campo sqlite_local identifica o banco usado nesta demonstração.”

Os POSTs também têm um seletor de exemplos: cadastro válido e teste inválido. Selecione o exemplo correspondente, ajuste o ID quando necessário e só então execute.

Passo 3: cadastrar a cidade

Em POST /cidades, clique em Try it out, substitua o exemplo inteiro e clique em Execute:

```
{"nome":"Sarandi"}
```

Esperado: Server response 201, com id_cidade e nome. Não confundir o Example Value da documentação com a resposta real do servidor.

“POST envia os dados para cadastro. O código 201 indica que o registro foi criado. O identificador da cidade foi gerado pelo banco.”

Anote o ID devolvido. Não assuma que é 1. Se o registro do ensaio já existir, um novo cadastro pode gerar outro ID; esta versão SQLite não impõe nome único.

Passo 4: cadastrar a linha

Em POST /linhas, use o ID realmente devolvido no campo id_cidade. O 1 abaixo é somente um exemplo:

```
{"codigo":"DEMO-20","nome":"Linha da apresentação","id_cidade":1}
```

Esperado: Server response 201 com cidade Sarandi e identificadores.

“A API verifica se a cidade existe antes de cadastrar a linha. Na resposta, retorna a referência e também o nome da cidade para facilitar a leitura.”

Passo 5: consultar

Execute GET /cidades e GET /linhas. Esperado: 200, listas contendo os cadastros realizados.

“Agora fazemos uma nova requisição de consulta. Os dados cadastrados aparecem na lista. Isso demonstra gravação e leitura no banco. A resposta é JSON, um formato que pode ser consumido por outra aplicação ou interface.”

“Agora o Gustavo vai demonstrar as validações.”

Parte 4 — Gustavo: testes de erro (cerca de 1 minuto e meio)

Passo 1: conferir que a cidade 999999 não existe

Na lista de GET /cidades, confirme que não há id_cidade 999999. Se existir, escolha outro inteiro positivo comprovadamente ausente.

“Uma API também precisa tratar dados incorretos. Vamos tentar cadastrar uma linha apontando para uma cidade inexistente.”

Passo 2: enviar o cadastro inválido

POST /linhas:

```
{"codigo":"TESTE-ERRO","nome":"Tentativa inválida","id_cidade":999999}
```

Esperado: 404, detail Cidade não encontrada.

“O 404 é o resultado esperado deste teste. A API identificou que a cidade não existe e interrompeu o cadastro. Essa mensagem é uma validação, não uma falha inesperada da apresentação.”

Passo 3: provar que não inseriu

Execute GET /linhas. A tentativa TESTE-ERRO não deve aparecer.

“A consulta confirma que a tentativa inválida não foi salva.”

Passo 4: testar texto vazio

POST /cidades:

```
{"nome":"   "}
```

Esperado: 422 com indicação do campo nome.

“A validação remove espaços das extremidades e rejeita o nome que ficou vazio. O código 422 informa que os dados recebidos não atendem às regras de entrada.”

“Agora a Jullyany vai explicar a fonte usada na parte de Ciência de Dados.”

Parte 5 — Jullyany: fonte pública e qualidade (cerca de 1 minuto e meio)

Passo 1: abrir GET /ia/dados-publicos/fonte

“Buscamos uma fonte pública para não criar números de transporte. Selecionamos um snapshot GTFS comunitário de Curitiba, publicado pelo projeto URBS-GTFS e catalogado na Mobility Database. O publicador declara usar dados abertos URBS espelhados pelo C3SL da UFPR. Nosso grupo consultou o arquivo publicado e sua documentação; não fez a coleta original.

O feed declara o período de 28 de setembro a 5 de outubro de 2026. Portanto é uma base histórica de Curitiba, não a programação atual de Maringá ou Sarandi e não um feed oficial da URBS. Aplicamos a técnica de análise do TRANSURBAN a esse estudo externo.”

Passo 2: abrir GET /ia/dados-publicos/qualidade

Indique registros_lidos, ids_programacao_distintos, ausentes_por_coluna, ids_duplicados e hashes_originais_conferem.

“O CSV foi produzido relacionando os registros de programação com os nomes das linhas e a operadora. Mantivemos os identificadores originais e a fonte. Cada linha do CSV é um registro de programação do GTFS convertido, não uma viagem comprovadamente realizada.

A verificação confere campos vazios, identificadores repetidos, referências e integridade dos arquivos. Os indicadores são bloqueados se o snapshot divergir ou tiver problemas nas regras verificadas. Não preenchemos dados ausentes com números inventados.

Não usamos os horários de chegada desse conversor para calcular atrasos ou duração. Agora a Camila vai explicar os indicadores que essa fonte permite calcular.”

A qualidade verifica o snapshot: não certifica a operação nem o conversor externo. Os arquivos originais incluídos e os hashes permitem conferir a derivação do CSV.

Parte 6 — Camila: números publicados, contagens e agrupamentos (cerca de 1 minuto e meio)

Passo 1: executar GET /ia/dados-publicos/resumo

“A primeira pergunta é: quantos registros de programação e códigos de linha existem nesse arquivo? Encontramos 71.086 registros e 309 códigos de linha.

Contamos quantos registros pertencem a cada linha. A soma das contagens reconcilia os 71.086 registros. A média dessas contagens é aproximadamente 230,05 registros por linha: 71.086 dividido por 309. A mediana das 309 contagens é 162.

Essas medidas são quantidades de registros, não minutos. Elas somam as categorias de serviço do arquivo e não representam uma média diária de ônibus ou viagens realizadas.”

Passo 2: executar GET /ia/dados-publicos/agrupamentos/linha

No Swagger, abra GET /ia/dados-publicos/agrupamentos/{dimensao}, clique Try it out, selecione linha, preencha limite com 5 e clique Execute. A resposta mostra cinco grupos, total_grupos 309 e o total completo separado da soma dos grupos exibidos. O limite é só de exibição; não muda o cálculo do resumo.

“A segunda pergunta é como esses registros se distribuem pelas linhas. Para facilitar a leitura no telão, exibimos cinco grupos de um total de 309. O agrupamento conserva o código e o nome publicados e informa a quantidade de registros e categorias de serviço presentes. A soma dos cinco grupos exibidos aparece separada do total completo.”

Não chamar o máximo de “linha mais movimentada”: não temos passageiros, veículos ou demanda observada.

Passo 3: repetir com dimensao servico

Deixe limite vazio ou mantenha 5: só existem três categorias.

“A terceira pergunta é como os registros se distribuem por categoria de serviço. Temos 33.425 para dias úteis, 20.754 para sábado e 16.907 para domingo. A soma dessas categorias também é 71.086.

A soma é do arquivo de programação, não o total de viagens realizadas em um dia ou semana. Os campos de viagens realizadas e atraso médio aparecem como null porque essas medidas não estão disponíveis. Null não significa zero.

Não há previsão de chegada nem IA preditiva nesta etapa. Agora a Ana Carolina vai concluir.”

Parte 7 — Ana Carolina: evidências e conclusão (cerca de 40 segundos)

“Demonstramos cadastro, consulta, relacionamento entre tabelas e validação de dados. Na parte de Ciência de Dados, mostramos a origem pública do CSV de Curitiba, a qualidade e os indicadores que ele permite calcular.

O projeto inclui testes automatizados e um gerador que confere 22 requisições HTTP e salva suas respostas. O código e as evidências estão versionados no GitHub na branch da entrega. A execução ao vivo de hoje usa o banco local.

A conexão da API com o Supabase continua como pendência. Não apresentamos login, GPS ou pagamentos reais nesta API. Obrigada pela atenção. Estamos disponíveis para perguntas.”

Só diga que os testes passaram no notebook da apresentação depois de executá-los nele. Nesta revisão são 29 testes e 22 requisições do gerador; evidências mais antigas registram versões anteriores. As medidas de transporte vêm do arquivo público, não dos testes da API.

5. O que demonstramos de DS e de IA

Área ou disciplina	Evidência concreta nesta entrega
Análise e Métodos de Sistemas	Tema de mobilidade, entidades Cidade/Linha, organização de responsabilidades e requisitos de cadastro/consulta
Banco de Dados I e II	PK, FK, relacionamento um para muitos, gravação, leitura e consulta com nome relacionado
Lógica Computacional	Verificação de existência, rejeição de texto vazio e regras de validação e agrupamento
Programação Backend I	FastAPI, POST/GET, Pydantic, SQLAlchemy, sessões, códigos HTTP e testes
Programação Python I	Módulos, funções, dependências isoladas e processamento em Python
Ciência de Dados	CSV, unidade de análise, qualidade, contagens, soma, média, mediana e agrupamentos com Pandas
Inovação e Tecnologia	Aplicação ao tema de mobilidade e disponibilização de indicadores para uma interface futura

Essa tabela relaciona conteúdos demonstrados; não substitui critérios específicos de correção do professor. O backend escolar tem duas entidades principais; não afirmar que todo o sistema de transporte está implementado.

6. Perguntas prováveis

O que é API? Uma interface com regras de comunicação. Neste projeto recebe requisições HTTP e devolve respostas JSON para acesso às funções do backend.

Swagger é a API? É uma interface de documentação e teste. As rotas continuam acessíveis por outros clientes HTTP.

GET e POST? GET consulta; nossos POSTs cadastram registros.

Por que voltar o nome da cidade se já tem o ID? O ID mantém a referência; o nome torna a resposta compreensível.

Model e schema? Model representa as tabelas e relações; schema valida os dados HTTP e descreve a resposta.

Por que venv? Para isolar bibliotecas e reproduzir as dependências sem misturá-las com outros projetos.

Por que não tem .env? No modo de apresentação, o inicializador define explicitamente SQLite local no processo. Outros modos podem ler a conexão privada no .env. Não é necessário projetá-lo nem distribuí-lo.

Onde ficam os cadastros? Em apresentacao_local.db no notebook que executa a API. Uma consulta após reiniciar confere a persistência no mesmo arquivo. O banco não é compartilhado automaticamente entre notebooks.

O Supabase está conectado? Ainda não foi comprovada a conexão Python. A estrutura foi conferida, mas a demonstração usa SQLite. Não confundir o editor SQL acessível pelo navegador com acesso PostgreSQL pelo Python.

A fonte é oficial e atual? Não. É um feed comunitário público, catalogado como não oficial, com período declarado de 28/09–05/10/2026. A origem primária URBS/C3SL é declarada pelo publicador; não afirmar coleta direta ou validação da URBS.

Por que Curitiba se o projeto é Maringá/Sarandi? Não obtivemos uma base local verificável nesta preparação. Usamos Curitiba como estudo externo para demonstrar o processamento sem fabricar registros locais. Para operar nas cidades do projeto, será necessário obter e validar uma fonte local.

De onde vieram os números de 7,57 e 8 minutos do roteiro antigo? Eram cálculos de uma base sintética de teste. Não são atrasos medidos e não são os indicadores desta apresentação.

Qual a fonte? O endpoint /ia/dados-publicos/fonte mostra o download, o publicador, o catálogo MDB-3225, o período, hashes e limitações. Os arquivos e o script de importação estão no repositório.

Por que usar SQLite? É um banco relacional local adequado à demonstração sem depender de acesso ao banco remoto. Não estamos afirmando que representa a implantação final de produção.

Os 71.086 registros são viagens realizadas? Não. São registros trip_id da programação GTFS convertida. Sem observações de operação, viagens_realizadas é null. Não contamos ônibus únicos ou passageiros.

Esses dados mostram atrasos reais? Não. Não temos tempos observados para isso. A soma e a média apresentadas são de contagens de registros de programação por linha, não de minutos.

O cadastro de uma linha altera os indicadores? Não. O cadastro DS usa SQLite; a análise lê o snapshot público externo de Curitiba. São fontes independentes e não existe sincronização automática.

Tem IA treinada? Nesta avaliação há Ciência de Dados descritiva; não há modelo preditivo treinado.

A análise prova o motivo do atraso? Não. Agrupamentos podem mostrar associações; não comprovam causalidade.

Tudo está na main e publicado na internet? A entrega está na branch indicada. Branch publicada não significa merge na main, API hospedada ou site em produção.

O frontend de passageiro usa esta API? O painel servido em /painel/ usa suas rotas. O protótipo React/Vite da escola e o site V2 são versões diferentes; não afirmar integração sem verificar.

7. Se algo não sair como esperado

- 404 no teste de cidade inexistente e 422 no teste de nome vazio são resultados esperados. Expliquem isso antes e depois da execução.

- Se a API não responder, confira o terminal. Não feche a janela do servidor nem pressione Ctrl+C durante o teste sem intenção de reiniciar.

- Se a porta estiver ocupada, não encerre processos desconhecidos. Inicie com --port 8001 e use 8001 no navegador.

- Se o Swagger estiver em branco, confira se é a cópia atual com swagger_static e se a API está funcionando. Evite tentar consertar bibliotecas no palco.

- Se aparecer outro resultado ou erro inesperado, não invente uma explicação. Diga: “Vamos conferir a execução; temos as respostas dos testes anteriores como apoio.” Use as evidências, identificando de onde vieram.

- Sem internet, não tentar Supabase, GitHub ou instalação. Com dependências e Swagger locais já preparados, a demonstração SQLite não depende desses serviços.

- Se o projetor não exibir, peça ajuda ao responsável pelo equipamento. A API funcionar e o vídeo projetar são verificações diferentes.

- Se o notebook não puder ser conectado, só use o computador do auditório depois de preparar e testar nele. Não existe garantia de funcionamento por apenas copiar um ambiente virtual de outro sistema.

8. Conferência final do líder

Antes de chamar a equipe: notebook carregando; cabo/adaptador conferidos; projeção legível; notificações silenciadas; API iniciada; /health com sqlite_local; Swagger carregado; snapshot público com 71.086 registros e origem aberta em uma aba; ID da cidade anotado; exemplos prontos; falas distribuídas; backup no pendrive. Não mudar o código depois do ensaio sem repetir a verificação afetada.

Esta orientação reduz riscos; não promete ausência de falhas em equipamento que ainda não foi testado. A validação final precisa ocorrer no notebook e na ligação com o projetor que serão usados.